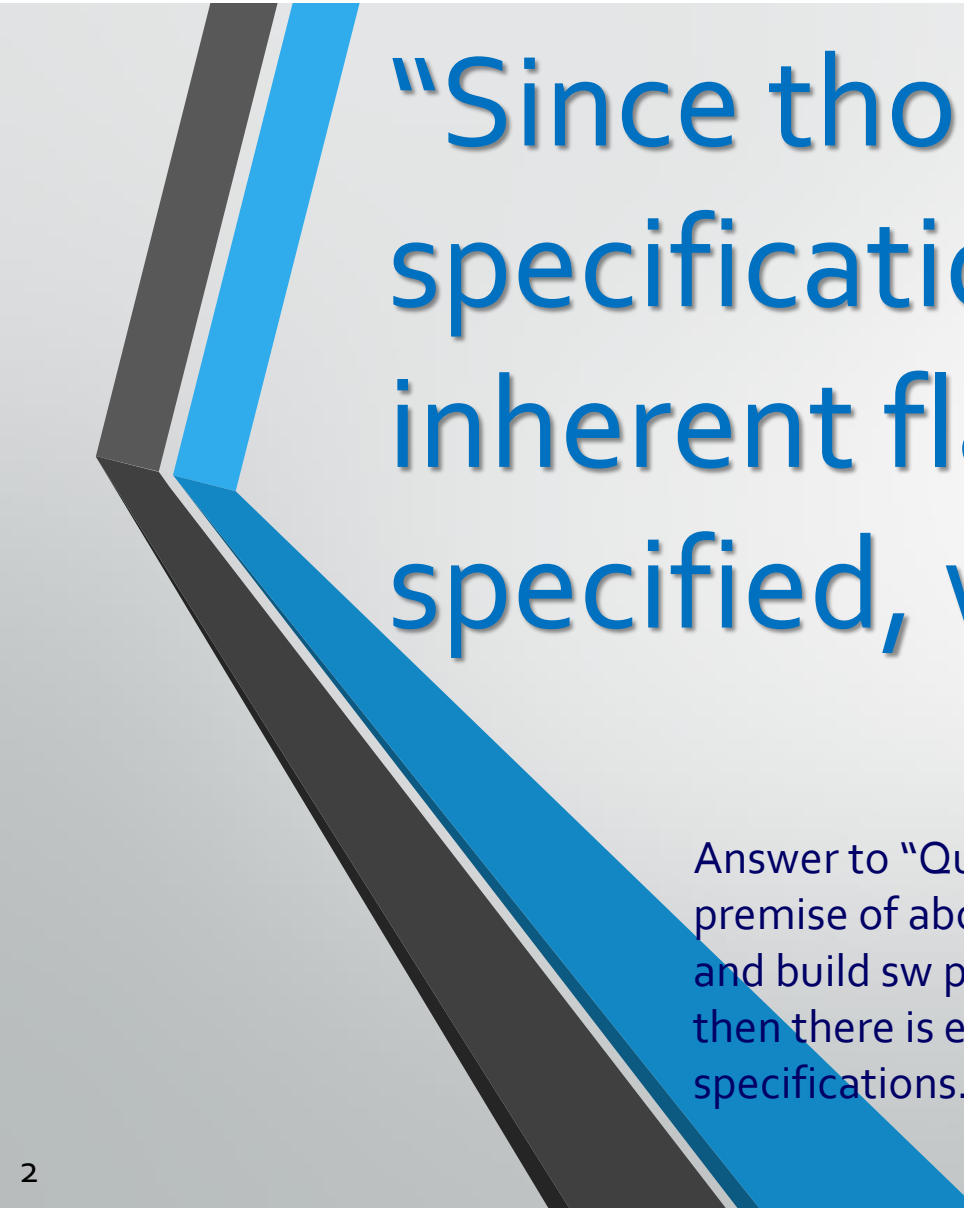


# ICT2113 SOFTWARE REQRS: NOW & NEAR FUTURE

## 8. Validating – QC SRS by Customer-Vendor & QA RE

Prof Kevin



# “Since thorough audits of specifications can’t detect inherent flaws in what is specified, what to do?”

Prof Kevin, 2023

Answer to “Quality”, second of ‘four horsemen of reqrs apocalypse’ (and premise of above question) is to **proceed** with ‘reasonable manifestation’, and build sw per SDM. If reqrs specification and its process **pass audits**, then there is every likelihood that sw will be **faithful semblance** of its specifications. After that, let customer decide what comes next.

# Agenda: QC for SRS & QA for RE

1. Previous week's learning.
2. Today's learning outcomes.
3. Video – “Validating SRS overview”.
4. Today's new learning material.
5. Active learning – Execution plan.
6. Epilogue.

# Previous week's learning

Six slides

Remember **why**  
review of previous  
week's learning  
**important!**

Strengthen memory  
retention.

Highlight knowledge  
gaps.

Prepare for acquiring  
new material (per  
**iterative** learning  
approach in HE insti-  
tutions).

# Specifying is 3<sup>rd</sup> in RE; vendor goes solo

Writing sw reqrs spec (SRS) that comprises base reqrs per agreed standards, and codified 'development provisions' best practices, i.e., latest technology, sw dev quality model, iRTM & change mitigation strategy, all needed by vendor in post-reqrs stages of SDLC.

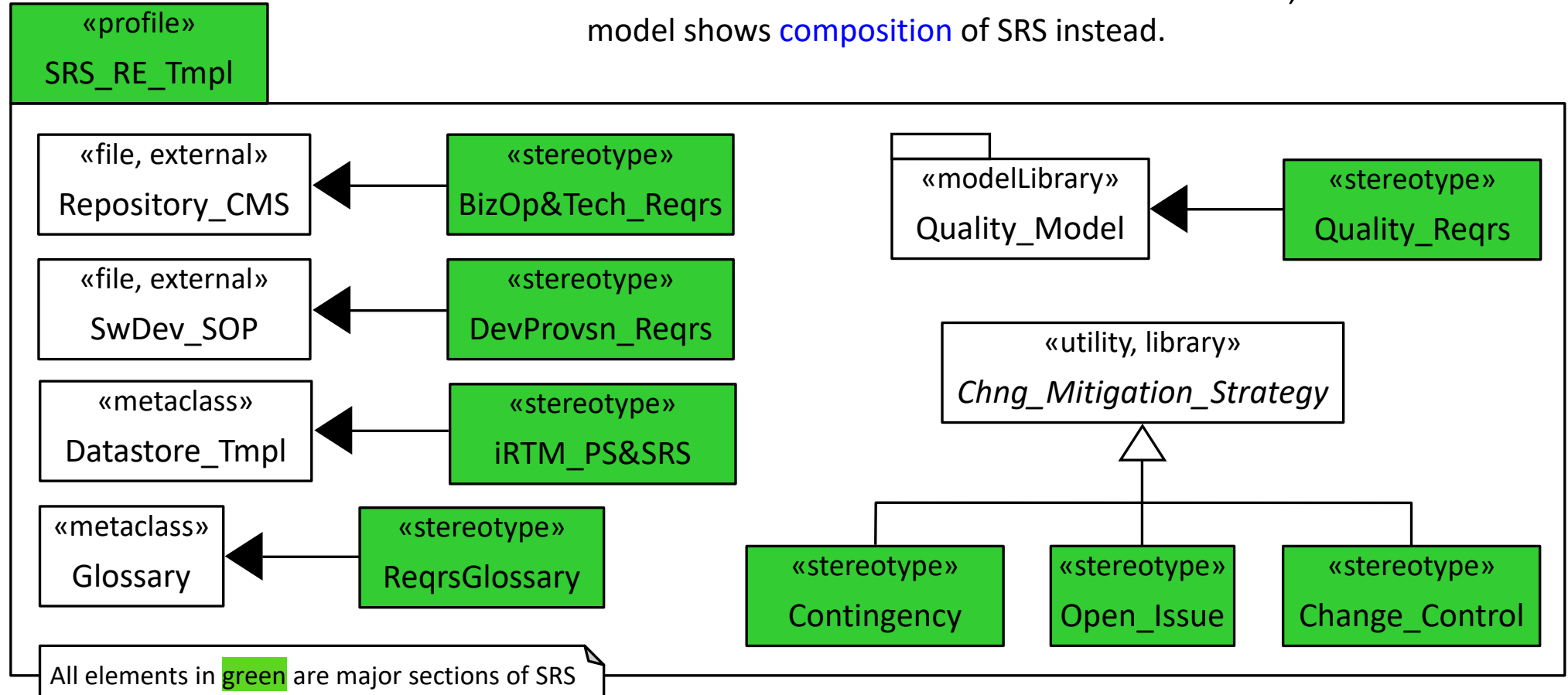
In Analysing method, vendor learns a) developing sw **not** customer's **core competency** (otherwise external vendor not contracted), and b) customer **demands** new sw product give it edge against competitors. Presume customer chose vendor because it proved it has **necessary expertise** to develop desired sw. Specifying method is right time for vendor to showcase that expertise.

Typically, vendor wants to build **superior** sw product envied by market players alike. Ergo, vendor adds **development provisions** c) **latest technology** that experience proves is "right" for customer's needs, d) proprietary **quality model** for sw development, e) its **reqrs-to-deploy tracing matrix**, and f) **change mitigation strategy**, into SRS to activate its proprietary experience and expertise in kind of sw development that it **believes** customer needs.

Summary of  
factsheet of  
fundamental  
ideas about  
Specifying  
method

# Model of SRS shows its content & origination

Since method's behaviour is at vendor's discretion, model shows **composition** of SRS instead.



(UML Package diagram)

# Repository's template insufficient for SRS

**Stock template** in Repository holds only **regulated-revision reqrs**, but does not cater clause-wise for **development provisions** because they can be determined only after Analysing. So, new expanded template is needed for development provisions in **reqrs clause form**. Stock template is **merged** into final template so as to avoid reinventing codification already established for regulated-revision reqrs. **Four** choices for expanded template:

1. Use template supported by your **company's IT** department, with standard boilerplate clauses for promoting company's details and brand.
2. Purchase template supported by **major standards agencies** (see right).
3. Co-opt template from thousands **available on Internet**,
4. Assemble own **custom template** from parts of various others, a.k.a., "Frankenstein template".

With so many reasonable choices, building template from scratch is not sensible.



# McCall's quality model

Brief definitions of McCall's quality features and perspectives applied to [new sw](#).

O  
p  
e  
r  
a  
t  
i  
o  
n  
  
T  
r  
a  
n  
s  
i  
t  
i  
o  
n

1. Correctness – extent no discrepancies of same data wherever it exists concurrently.
2. Efficiency – smallness of: code lines + utilization of time, memory, hardware & resources.
3. Integrity – extent unauthorized access or attack is avoided or withstood.
4. Reliability – extent functions perform in specified conditions without failure.
5. Usability – smallness of time to learn, operate & control in specified usage scenarios.
6. Flexibility – extent perform in unusual conditions or variances in interface or int processes.
7. Maintainability – extent continuous monitoring, repairing & upgrading for max useful life.
8. Testability – extent allows user checks for problems or does self-diagnosis and records.
9. Interoperability – extent exchanges data via standard protocols and formats.
10. Portability – smallness of time and effort to adapt to new OS, compiler and hw.
11. Reusability – extent its parts can be installed and used in another program.

R  
e  
v  
i  
s  
i  
o  
n

# Codifying and inceptive RTM

Requirements traceability matrix (RTM); first version is ‘inceptive RTM’ or iRTM.

- ▶ **Umbrella** document “living” for entire SDLC, **iterative** with successive versions, and **linker** of reqrs to all project’s data through to user acceptance testing, ensuring all reqrs are accounted for across SDLC.
- ▶ More than checklist, a) “chain of custody” of reqrs from PS to SRS to Design Spec to Coding Spec and so on, and b) dynamic adapter of any documents in project, e.g., reqrs, models, links to other docs, and even code.
- ▶ While there is no record when first RTM was introduced, concept of ‘reqrs traceability’ was first introduced in publication by Penedo & Stuckle in 1985.
- ▶ Integrity of RTM relies entirely upon goodness of codifying in SRS.

## Codifying

- ▶ Assigning **unique identifier** (**numeric** or **alphanumeric string**) for every reqr SRS.
- ▶ Key factor is how **far into reqrs statements should team take it**. Single statements are no-brainer. However a) **compound** statements (two or more statements linked together by conjunction, such as "and", "or"), b) **lists** such as bullet points, and c) **Elements in models**? For all of these, codifying is difficult decision.
- ▶ recommended **not** to use unique outline numbering used for table of contents for codifying, as this may be changed often, and even minor changes to contents can break codification in RTM.

# Reqs change mitigation strategies

Change to some SRS content is inevitable, due to changes in industry, market, or government **regulations**. If changes result in additional work unfunded by governing contract, this **weakens** vendor's business. Hence, vendor should anticipate change and include plans in SRS to **manage & mitigate** changes.

1. **Anticipatory contingency reqrs.** These are **pre-agreed** by stakeholders and quietly tucked away in SRS, only to be introduced into vendor's development plans when either **specific conditions have been met** or **events have occurred**. E.g.#1, new government regulations are released either early or postponed; e.g.#2, new business unit created and warrant change to info systems schema.
2. **Open issues.** Reqs still **not agreed** between Analysing team and stakeholders within time allotted are **placed into SRS** for **continuing negotiations** on separate channel after development has commenced. Kill date should be stipulated for open issues not addressed and resolved during development.
3. **High-risk change control.** High-risk change control. This is **strict pre-approved** process that scrutinizes, vets, and constrains ad-hoc change proposals on pre-agreed reqrs with high risk of **severely damaging** project or product if changed. Also, changes can be deemed **invalid**, such as new reqrs, i.e., not existing in SRS.

# Today's learning outcomes

One slide

Remember **why**  
knowing learning  
outcomes **important!**

Roadmap of learning  
goals & expectations.

Encourages develop-  
ing preparation &  
study strategies.

Promote engaging in  
learning process.

Facilitate managing  
learning material &  
effort.

# LO#8: QC for SRS & QA for RE

After today's lesson, student will be able to explain ...

- ❑ ... Validating method, and how it incorporates QC & QA.
- ❑ ... use of QC to “catch” three kinds of errors — authorship indiscriminate, latent contradictions, omission of topics — via min six investigation techniques.
- ❑ ... use of QA to “catch” ‘failure to strictly comply with RE(-) methods’ error via min three investigation techniques.
- ❑ ... what's next in RCMs and what alternatives are there to RE.

Ensure you find and remember where in Learning Content is all info essential for achievement of every Learning Outcome.

Don't let this fester! If unsure, please come see me.

# Video

**One slide & 4:17 mins clip**

**Clip08 Validating SRS Overview+Eg**

# Things to keep in mind before viewing

It is hard to find videos that truly conform to our module's more up-to-dateness and adherence to IEEE's **real** conceptualization of Validating method. Today's video gets close, but there are still **overlaps** with Analysing method. Nevertheless, I think it helps your learning to recognize these overlaps immediately, and then bring them up for discussion with teacher or TA.

On professional note, I highlight his example of **unmoveable date** for going live, although it comes to Validating because it wasn't addressed earlier. Can you tell me **when you should have flagged it, and how you would handle it in SRS?**

Look forward to discussing these with some of you who are starting to master our learning objectives.

Any questions? Signal when you understand, and are ready to begin.

# Today's **new learning material**

**Twelve slides**

# Validating is 4<sup>th</sup> in RE; vendor goes solo

Applying **quality control** measures onto SRS, and conducting **quality assurance** audit of vendor's conduct of Analysing, Specifying, and Validating against **contractual and professional standards**, to ensure that vendor has **right SRS** produced **right**.

**Vendor solely responsible.** Regulated-revision by vendor renders **customer's culpability negligible**.

**Independent audits.** Due to SRS's significance, complexity & high risk of error, Validating method is audits conducted by **independent** professional **auditors**; customer & vendor do only **reviews**.

**Two things are audited,** a) **SRS**, and b) **RE(-) process** that makes it. Former is quality control (QC), and latter is quality assurance (QA). Typically, QC after SRS completed, and QA before SRS is started. Our module is exception as we study QC & QA together in Validating method.

**Different auditors for QC & QA.** This is because **error mappings** of product and process are entirely different. QC is **physical inspection** of **goods**, while QA is **desktop analysis** of activity's **paper trail**.

**Auditors only write reports.** Reports **identify** errors detected and **explain** appropriate rectifications, but effective writing of corrections to SRS are purview of vendor with customer agreement.

# Broad terms for QC and QA auditing

QC and QA are two methods of high-level process of Quality Management (QM) that specify techniques and tools for establishing quality across all forms of manmade goods and services providers. QC and QA are designed to root out defects; they differ in their targets and extent of actions.

Target of QC is workproduct generated by process, in other words, defect detection not prevention. QC procedures include batch inspection, product sampling, statistical analysis, and level 1-3 testing. Elimination of defects is not normally done, especially with manufactured and specialty goods.

Target of QA is process producing workproduct. This broadly entails documentation, suppliers, engineer & manager training, and workproduct changes, to ensure compliance with standards such as ISO/IEC 25010. Generally, QA takes place prior to inception of process. Categories of QA are defect prevention, continuous improvement, human resource management, and application of risk-based approaches. Its procedures include investigative reporting, audits of internal auxiliary processes, planned system reviews, and spot checks.

**Start active learning Part 1, while  
those needing it take quick break**

**Quick  
break &  
Part 1**

**300 sec countdown – ends when fully filled**

**Let's  
continue  
with LS**

# Validating method basics: SRS

1. Vendor's **Project Manager (PM)** in-charge of QC audit, directing Reqs team to support QC auditor, including access to SRS and on-site workspace.
2. Before QC independent auditor arrives, vendor & customer cooperate to **review** SRS and **correct** any high profile errors, such as:
  - ▶ missing essential reqrs, plus any agreed edits per Analysing sessions; customer.
  - ▶ change mitigation, codification coherence, quality model, iRTM & typos; vendor.
3. Auditor's '**Execution plan**', including T&Cs of auditing SRS, access control (security clearance), audit team size & roles, kinds of errors being pursued, and reporting, is provided to PM for approval (may need final amendments).
4. Auditor ensure appropriate **security controls** of sensitive info and integrity of SRS is maintained throughout audit.
5. Following '**Post-op report**', PM advises Reqs team to make necessary enhancements. Reqs team rarely need customer involvement at this time.

For **labtorial**,

1. Each tm is independent auditor for **another** tm.
2. Tm being audited gives **their SRS** to tm doing auditing.
3. Instructions for tm size, roles & tasks set out in **lab guide**.
4. SRS amending **not** necessary.

# Validating method basics: RE(-) process

1. Vendor's **C-staff** in-charge of QA audit, e.g., CIT or CIO, directing Reqs team to support QA auditor, including plans, reports, checklists, meeting decisions & historical data records for Analysing, Specifying, and Validating.
2. Typically, key business processes are constantly undergoing QA audits for maintaining quality. There may be **regular audits**, and even **regular auditors**. Regular audits likely conducted by **one** seasoned auditor.
3. Auditor's '**Execution plan**', including T&Cs of auditing process methods, access control (security clearance), kinds of errors being pursued, and reporting, is provided to C-staff for approval (may need amendments).
4. Auditor ensure very stringent **security controls** of sensitive info and integrity of RE(-) process is maintained throughout audit.
5. Following '**Post-op report**', C-staff may open QM project plan, and direct company's department heads to make their own parts of QM project plan for implementing necessary enhancements to their departments.

For **labtorial**,

1. Within each QC auditor tm, **one** member is QA auditor.
2. QA auditor audits their own tm's QC, plus other tm's RE methods.
3. Instructions for QA tasks set out in **lab guide**.
4. There is **no** amending after report done.

# What errors left to consign to QC and QA?

SRS should **already be in great shape** after conduct of Analysing and Specifying methods. Reqs are effectively written with Mag7 & R-tree. All customer's incongruent reqs are repaired by regulated-revision using IRC. Vendor's & customer's models are revised from "paper napkin" into professional standards-compliant forms. So, what errors may possibly be left to consign to QC and QA? These are **four** possibilities:

1. **Indiscriminate side effects of authorship.** Authors can't see their own errors in their writings because their brains automatically overemphasize meanings that were **intended**, rather than notice the exact text by which those meanings are expressed.
2. **Omissions of topics.** These errors only become apparent when document is **comprehended** in its completeness that any holes can be noticed.
3. **Latent contradictions.** These lie hidden within details of huge quantity of reqs & provisions. They become apparent when, likely by accident, contradictions are noticed when read **consecutively**. However, it is more likely one is read first, and other quite long after. Photographic memory helps.
4. **Failure to strictly comply with RE(-) methods.** Actions and tasks missed due to a) insufficient or invalid **evidence**, b) insufficient **resources** for jobs, c) insufficient of inadequate **management** tools, and d) inadequate **training**. Set aside any possibility of incompetence and malfeasance.

QC

QA

# Investigative techniques: Error QC#1

- A. Apply **personal checklist**. Auditors a) reuse earlier list, b) derive one from literature search, or c) follow applied template. Said **personalized** lists enable team detecting wider variety of errors.
- B. Separate **spellchecks**. Auditors do **own** spellcheck, extending to repeated words, reversed letters, incorrect pluralizing, plus grammar. Again, to detect wider variety of errors.
- C. Focus on one at a time. Auditor reads SRS **several** times, looking each time for only **one** type of error, e.g., sentence structures, word choices, spelling, and punctuation.
- D. Read document **backward**. (From last word, go right to left and bottom up.) This helps auditor to focus on **individual words** rather than on sentences or meaning.
- E. Personally read document **aloud**. This facilitates concentration on **spelling** of words.
- F. Ask colleague to read it **aloud**. Error detection is known to increase when passages are **heard** instead of read.
- G. Project document on screen. Reading via **different media** can make error instances stand-out. This also occurs when going from hard copy to softcopy, or vice-versa.
- H. Apply **dictionary & thesaurus**. If you are not sure of **right word**, Word Hippo is one of best word analysers on Internet.

No order to this list, nor is it mandatory for conducting all in list. Choose with discretion & manage wisely time on task.

# Investigative techniques: Errors QC#2 & #3

- A. Skim. Do entire SRS from **top-to-bottom** to identify any **missing & different sections** in comparison to previous projects. Then go **bottom-to-top**, doing similar search. Slight sense of **discontinuity** means possible omission.
- B. Double-check. Dates, facts, figures, and proper names, for contradictions.
- C. Persona-based reading. This aims to detect both kinds of errors within large collection of reqrs & provisions by comparing SRS to known workflows from perspectives of persona/roles, e.g., ...
  - ▶ user – how usable and useful are described functions and features.
  - ▶ department manager – how inclusive, timely, and useful are decision-making support functions.
  - ▶ customer PM – are there any additional costs, e.g., subscriptions, not in contract.
  - ▶ sw architect – how complete are details for setting up APIs and deriving internal components with all relevant performance properties.
  - ▶ sw developer – what information do I still need in order to code ... (next page)

They take lots of effort to prepare, but they are **effective**.

Conduct **singly, combined, or interleaved**.

Our module's slide deck is checked using **discontinuity** approach to find omissions, and **double-checked** for latent contradicts.

# Investigative techniques: ... QC#2 & #3 (cont)

... functions and features, especially GUIs and database elements.

- ▶ vendor PM – has necessary information for licencing of all required listed third party sw been provided.
- ▶ tester – how effective are data items for deriving useful test cases.

**D. Scenario-based reading.** This also aims to detect both kinds of errors within large group of reqrs & provisions by virtue of incongruencies between SRS and particular scenarios involving biz ops processes, e.g., ...

- ▶ operations.
- ▶ HR.
- ▶ finance accounting and treasury.
- ▶ warehousing.
- ▶ sales and distribution.
- ▶ supply chain.

**E. Assumption reading.** This aims to detect assumptions that contradict reqrs & provisions of SRS, or have been made redundant by being reworked into reqrs & provisions that came after assumptions.

# Investigative techniques: Error QA#4

- A. **Apply tasklist**. Prepare **tailored** investigation tasklist from **mining** supplied policy docs, then, apply this to supplied **records** of action/task conception, formulation, assignment, conduct, and debrief, in order to detect obvious errors. Records include work orders, meeting minutes, journals, SITREPs, expense claims, and phone GPS location tracking reports. (Conducted similarly to **desktop study**.)
- B. **Do field study**. Understand operational situation and identify error causes by a) **observing** (unobtrusively) participant in action, b) **querying** about purpose and conduct of instructions/ guidelines, and c) **interviewing** (last) to reveal deeper details. (Applicability depends on fortuitous timing with ongoing operations.)
- C. **Do data imputation**. **Replacing missing** records with estimated or plausible items to **reconstruct** fragmented workflows. This builds understanding of **nature of workflow** breaks and determining if such breaks are random (ad hoc) or systemic.
- D. **Cross-referencing alternate sources**. Build predictive models of workflow to reveal **anomalies** in actual conduct of actions/tasks along with integrity of existing record set.
- E. **Inject invalid record items**. Observe their **effect** on operations & management actions/tasks, e.g., validity confirmed or not, as well as benign or harmful; integrity issues?

Auditor can be **single** or team. Effectiveness depends on customer supplying right right docs, and enough time for analyzing everything in depth.

# Prognosis for detecting errors in Validating

Errors consigned to QC are **most challenging** to detect, being arbitrary, insidious, and steeped in **specialist semantics**. Errors consigned to QA are fairly easy to detect in well-defined workflows and policy directives.

In **QC**, most headway made with '1.Authorship errors'; perhaps around **85%** of error instances might be detected and corrected. In contrast, it could be near **zero** detection for '3.LatentContradictions'. '2.OmissionOfTopics' is probably **10-50%**.

Conversely in **QA**, detecting **lack of justification** for findings, derivations, conclusions, or time spent on job, will approach **95%**.

Passing validation **doesn't guarantee** that SRS is still valid at later point in time, because stakeholders gain additional insight and familiarity with their new sw. This may be especially true if ...

- ▶ ... there are lots of innovative ideas and technology imbued into new sw.
- ▶ ... there is significant gain of knowledge required for new sw.
- ▶ ... development of new sw is of long-duration.

# So, Validating method is over; what's next?

Crucially, RE is **completed** and reqrs are releasable; in SDM, vendor takes over.

- ▶ PM releases SRS to development team (dev tm), and announces **start of sw dev project** with SDLC plan debrief to whole tm.
- ▶ Vendor's reqrs analysis team **absorbed** into PM's dev tm, but retain customer tm's contacts to action SRS change mitigation strategies. First task is **debrief** dev tm's architect & designers pouring over SRS details to understand capabilities and conditions therein.

What are your immediate takeaways concerning RE?

- ▶ **Heavyweight** RCM is **ponderous**. Sure, that's supposed assure quality of resulting sw, but is it effective? Arguably, majority of resources expended in RE not directly writing reqrs clauses.
- ▶ No doubt that technology advancements will **outpace RE**, if they haven't already. This disadvantages any customer wanting to get edge over its competition by developing new not existing sw.
- ▶ There are many cases of sw dev project **failures**, reqrs take so **long to deliver**, too many resources expended in **non-productive** activities. Surely, there are alternatives to RE?

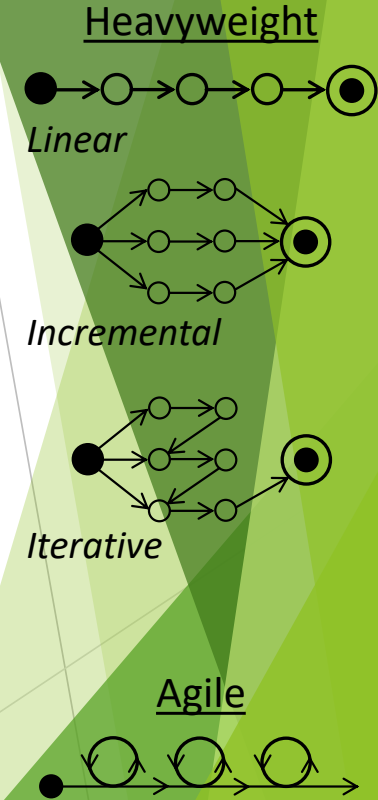
# ...Validating ... is over; what's next? (cont)

Any RE alternates that are also heavyweight?

- ▶ (See upper right) **Incremental** and **iterative** SDLCs can partition RCM allowing concurrent conduct, and reduced time. (*Car parts separately or build onto frame; divide & conquer.*)
- ▶ Pre-built **Enterprise resource planning (ERP)** sw is on-premises and can be deployed **as is** or **customized** to fit customer's special needs. RCM reduced to determining which of ERP's inherent functionality customer wants a) activated as is, and b) customized. (*Kit car.*)
- ▶ **Hosted** sw is off-premises used **as is**. RCM reduced to determining if enough of hosted sw meets customer's special needs at reasonable price. (*Leasing a car.*)

Lightweight, in particular Agile (see lower right), is way forward.

- ▶ **Advantages** include flexibility, rapid delivery, higher customer satisfaction through frequent feedback, enhanced tm collaboration, and improved adaptability & response time to changes.
- ▶ **Disadvantages** include challenges with cultural shifts in rigid organizations, lack of documentation, difficulty predicting costs & timelines, and potential for scope creep with burnout due to constant interaction.
- ▶ Our module shifts to studying **two** lightweight RCMs: **User stories** and **UseCase2.0**.



# Active learning 1-2

Two slides

# These are instructions for session

There are **two parts** to today's Active Learning session:

Part 1 – Explain how IT vendor's QC might be expanded from defect detection into defect prevention, and why its challenges outweigh its benefits?

Part 2 – Propose your team's draft execution plan for QC audit in labtorial?

After slide 17, we start doing Part 1. After completion, we return to slide 18 to finish last part of Learning Content. After that, we start Part 2 of session.

**For Part 1.** Consider status quo of QC audit for sw is, for sound reasons, defect detection. Nevertheless, teams do what-if **exploration** of it moving into prevention, and evaluate negative impact of that on IT vendor's operations.

Teams have **five** mins to come up with their answers. Once time is up, teams be prepared to **share** their results if called by teacher. Feedback is available from teacher and peers.

Any questions? Wait for teacher's signal to begin Part 1.

# These are instructions for session (cont)

**For Part 2.** Each team assumes role of QC auditor, and plans how to conduct QC of SRS **for real** in labtorial, i.e., '**execution plan**'. Includes a) list of error mapping **techniques** to be used, b) **WBS** for each member, and c) real estimated **timeline** for member's taskings.

QC & QA investigative techniques provided in slides 22-25, and members assigned QC team generic roles per labtorial guide Appendix.

Notes: d) One member of each team transferred to QA audit. Hence QC actual team is less one member, and QC execution plan must show as much, and e) teams will audit **another team's SRS**, not their own. Allocation of which SRS is audited by each team decided by this week's labtorial.

Teams have **15** mins to come up with their plan. Once time is up, teams be prepared to **share** their results if called by teacher. Feedback is available from teacher and peers. After LS, **distribute** plan among members for further elaboration, and be prepared for discuss with teacher for feedback in this week's labtorial. Post notes.

Any questions? Wait for teacher's signal to begin Part 1.

# Epilogue

Two slides

Always be mindful that success of heavyweight sw development relies on “goodness” of SRS. But, it can’t tested for defects like can be done with code. Instead, it needs quality management methods, most commonly QC & QA. QC checks that SRS doesn’t misrepresent integral needs of stakeholders, and QA checks that process making SRS doesn’t harbour activities that enable misrepresentations to be embedded into SRS. Naturally, critical thinking and experience are “sine qua non”, so it’s prudent that professional auditors carry out Validating method. Maximizing assurance that SRS plus process making it, are both right and true, more than justifies time and cost of having professional auditing.

**“It’s not only a strong start and middle, but also no skimping at the end.”**

# My final words

- ▶ Contact me for ...
  - ▶ regular stuff via my work email [h51581@singaporetech.edu.sg](mailto:h51581@singaporetech.edu.sg).
  - ▶ more sensitive stuff, by Whatsapp [97761716](tel:97761716) or my personal email [johnmilton2000@gmail.com](mailto:johnmilton2000@gmail.com).
- ▶ Tomorrow's lab is 1<sup>st</sup> on subject of [Specifying](#). Hopefully, your Analysing was successful, and you have gathered all reqrs and models necessary for SRS in [repository](#). In preparation, ensure you are familiar with ...
  - ▶ ... [your repository with all "cleansed" reqrs from PS](#).
  - ▶ ... [your chosen template and quality model you will be using in your SRS](#).
  - ▶ ... [various RTM examples in Internet](#)
  - ▶ ... [chosen change mitigation strategies \(from active learning\)](#).
- ▶ Keep on working through [Glossary](#), four terms per week, and Google.
- ▶ Teachers take certain concepts being understood for [granted](#). If you are falling behind, see me or TA Ernest for [consultation](#).

Glossary & Google readings for next week.

- ❑ Web search on latest trends of reqrs capture in SE Asia.
- ❑ Agile development of sw.

# Any final thoughts?

If not ...

End of LS#8

Validating – QC SRS by Customer-Vendor & QA RE  
by Prof Kevin

ICT2113 Software Reqs: Now & Near Future